

# 报告生成平台-架构设计文档

## 报告生成平台架构设计文档

版本：v1.0

状态：草稿

架构形态：模块化单体架构

技术栈：Java / Spring Boot / MyBatis / MySQL / Redis / MinIO

适用范围：密码安全应用测评报告、水利行业文档、自定义报告生成场景

## 1. 文档概述

### 1.1 文档目的

本文档基于《报告生成平台产品需求文档 PRD》进行架构设计，明确报告生成平台的系统边界、模块划分、核心流程、数据存储、外部系统集成、安全机制、部署方案以及异常处理机制。

本文档用于指导后续详细设计、接口设计、数据库设计、编码实现、测试验证和系统部署。

### 1.2 系统定位

报告生成平台是一个基于智能体能力的专业文档生成系统，面向密码安全应用测评报告、水利行业文档、自定义报告等场景，提供从模板选择、参考文档上传、知识库检索、Agent 内容生成、报告渲染、在线预览、下载归档的完整能力。

系统本身不直接实现大模型能力和知识库检索能力，而是通过标准 HTTP 接口调用外部 Agent 服务和外部知识库服务。

### 1.3 架构目标

系统一期采用模块化单体架构，在一个 Spring Boot 应用内按照领域边界划分模块。

核心目标如下：

- 支持密码安全应用测评报告、水利文档和自定义报告生成。
- 支持模板上传、模板变量解析、模板选择和模板版本管理。
- 支持参考文档上传、解析、存储和关联。
- 支持调用外部 Agent 完成报告内容生成、合规审查和智能分析。

- 5. 支持调用外部知识库完成标准条款、行业规范和上下文增强检索。
- 6. 支持报告生成任务状态管理、日志追踪、失败重试和人工修订。
- 7. 支持报告文件存储、在线预览、下载、分享和历史版本管理。
- 8. 保证系统具备可扩展性、安全性、可观测性和稳定性。

## 2. 架构约束与设计原则

### 2.1 架构约束

| 约束项      | 说明                                                |
|----------|---------------------------------------------------|
| 后端技术栈    | Java 为主，采用 Spring Boot 构建后端应用                     |
| ORM 框架   | 使用 MyBatis / MyBatis-Plus 访问 MySQL                |
| 文件存储     | 使用 MinIO 存储模板、参考文档、生成报告、预览文件                      |
| 数据库      | 使用 MySQL 存储业务元数据、任务状态、模板信息、报告记录                   |
| 缓存与任务队列  | 使用 Redis 缓存热点数据，也可使用 Redis Stream / List 实现轻量异步任务 |
| Agent 能力 | 不在本系统内部实现，采用外部 HTTP API 调用                        |
| 知识库能力    | 不在本系统内部实现，采用外部知识库 HTTP API 调用                     |
| 部署形态     | 单体应用部署，可部署多个应用副本                                  |
| 用户认证     | 支持 SSO / JWT / RBAC 权限控制                          |

### 2.2 设计原则

#### 2.2.1 模块化单体

系统不是把所有代码混写在一起，而是在一个 Spring Boot 应用内部按照业务领域进行模块拆分。每个模块拥有独立的 Controller、Application Service、Domain、Repository、Mapper 和 DTO。

#### 2.2.2 领域边界清晰

系统按照业务能力划分为报告管理、报告生成项目、模板管理、文件管理、任务编排、Agent 适配、知识库适配、用户权限、审计日志等模块。  
模块之间通过应用服务接口进行交互，避免跨模块直接访问内部数据表和 Mapper。

### 2.2.3 外部能力适配隔离

外部 Agent 服务、外部知识库服务、MinIO、Redis、SSO 等系统能力统一通过 Adapter 适配层封装。业务模块不直接感知外部 API 的 URL、鉴权方式、请求格式和重试逻辑。

### 2.2.4 异步任务驱动

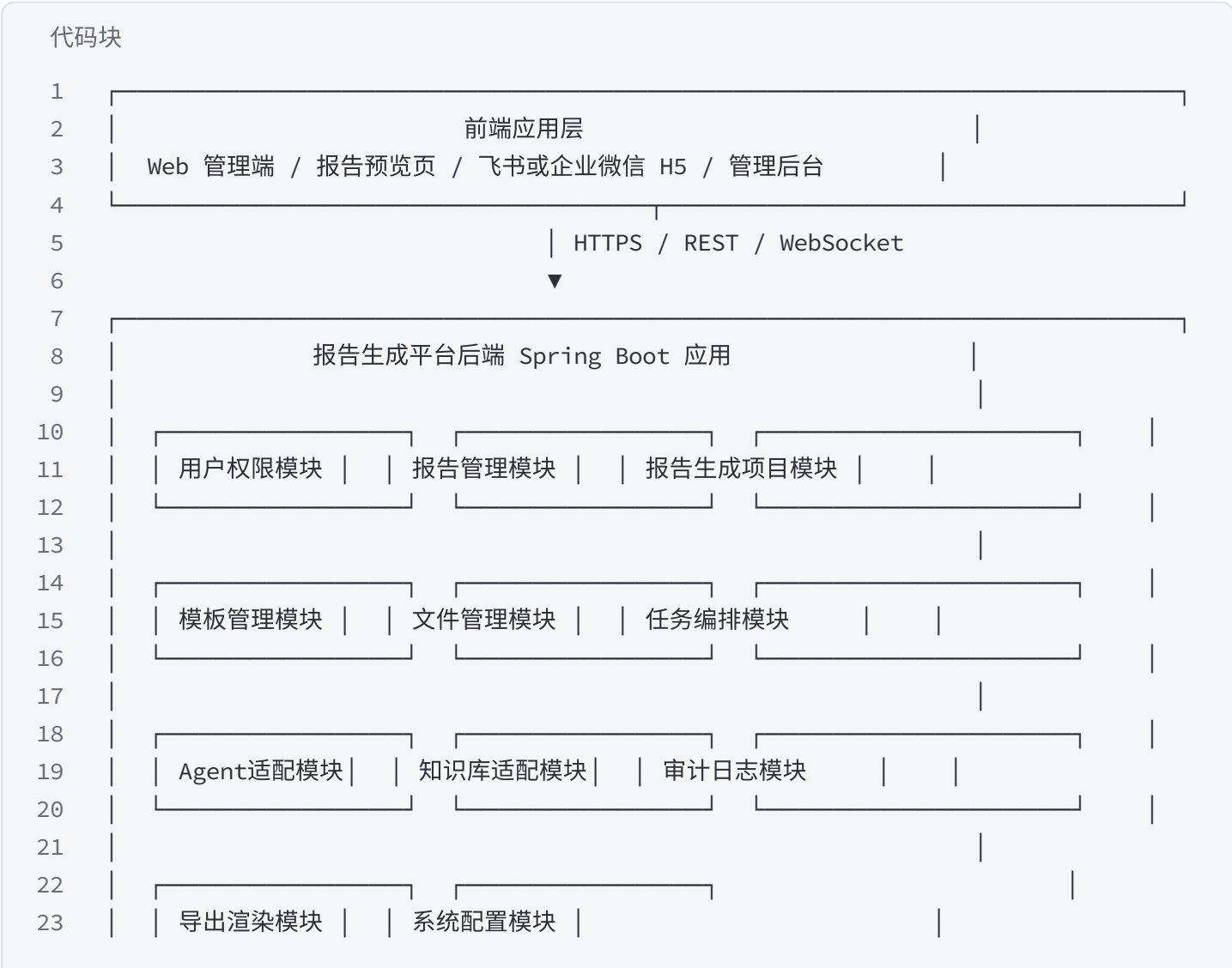
报告生成属于耗时任务，用户提交生成请求后，系统创建生成任务并异步执行。  
用户可以通过任务状态接口或 WebSocket 查看生成进度，避免接口长时间阻塞。

### 2.2.5 全链路可追踪

报告生成过程中的输入参数、模板版本、参考文档、知识库检索记录、Agent 调用记录、生成结果、人工修改记录都需要留痕，便于问题排查、审计追踪和质量优化。

## 3. 总体架构设计

### 3.1 系统总体架构图





### 3.2 架构说明

系统采用单体应用部署，后端主体为一个 Spring Boot 应用。

该应用内部按照业务领域划分模块，各模块之间保持清晰边界。

整体分为五层：

| 层级    | 说明                                       |
|-------|------------------------------------------|
| 前端应用层 | 提供项目创建、模板选择、报告预览、报告下载、后台管理等页面            |
| 接口层   | 提供 REST API、WebSocket API，负责参数校验和响应封装    |
| 应用服务层 | 编排业务用例，例如创建报告、提交生成任务、查询任务状态              |
| 领域模块层 | 管理报告、模板、任务、文件、权限、审计等核心业务对象               |
| 基础设施层 | 负责 MySQL、Redis、MinIO、外部 Agent、外部知识库等技术适配 |

## 4. 应用分层设计

### 4.1 分层结构

代码块

```
1  Controller 层
2      ↓
3  Application Service 应用服务层
4      ↓
5  Domain 领域层
6      ↓
7  Repository / Mapper 数据访问层
8      ↓
9  MySQL / Redis / MinIO / 外部 HTTP 服务
```

## 4.2 Controller 层

Controller 层负责对外提供 HTTP 接口，不直接编写复杂业务逻辑。

主要职责：

1. 接收前端请求。
2. 完成参数校验。
3. 获取当前登录用户信息。
4. 调用应用服务。
5. 返回统一响应结构。
6. 不直接调用 Mapper。
7. 不直接调用外部 Agent 和知识库。

## 4.3 Application Service 层

Application Service 层负责完整业务用例编排，是事务边界所在。

例如“提交报告生成任务”会涉及：

1. 校验用户权限。
2. 校验模板是否存在。
3. 校验参考文档是否有效。
4. 创建报告记录。
5. 创建报告版本。
6. 创建生成任务。
7. 写入任务队列。
8. 返回任务 ID。

## 4.4 Domain 领域层

Domain 层负责表达核心业务概念和业务规则。

典型领域对象包括：

| 领域对象             | 说明       |
|------------------|----------|
| Report           | 报告主体     |
| ReportVersion    | 报告版本     |
| ReportProject    | 报告生成项目   |
| Template         | 报告模板     |
| TemplateVariable | 模板变量     |
| GenerateTask     | 生成任务     |
| TaskStageLog     | 任务阶段日志   |
| AgentProfile     | Agent 配置 |
| FileObject       | 文件对象     |
| AuditLog         | 审计日志     |

领域层不直接依赖 Spring、MyBatis、MinIO SDK、HTTP Client 等技术框架。

## 4.5 Repository / Mapper 层

Repository 提供面向领域的持久化接口，Mapper 负责具体 SQL 实现。

约束如下：

- 1. Controller 禁止直接调用 Mapper。
- 2. 跨模块禁止直接调用对方 Mapper。
- 3. 写操作必须经过 Application Service。
- 4. 复杂查询可以放入 QueryService。
- 5. MyBatis XML 中维护复杂 SQL。

## 4.6 Infrastructure Adapter 层

Infrastructure Adapter 层负责封装外部系统和技术组件。

包括：

- 1. MinIO 文件上传、下载、预签名 URL。
- 2. Redis 缓存、分布式锁、任务队列。

- 3. 外部 Agent HTTP API。
- 4. 外部知识库 HTTP API。
- 5. SSO 用户认证。
- 6. Word / PDF / Markdown 渲染工具。

## 5. 后端模块设计

### 5.1 包结构设计

代码块

```
1  com.company.dgp
2  |—— DgpApplication.java
3  |
4  |—— common
5  |   |—— result
6  |   |—— exception
7  |   |—— config
8  |   |—— security
9  |   |—— util
10 |   └── enums
11 |
12 |—— auth
13 |   |—— controller
14 |   |—— application
15 |   |—— domain
16 |   |—— repository
17 |   |—— mapper
18 |   └── infra
19 |
20 |—— report
21 |   |—— controller
22 |   |—— application
23 |   |—— domain
24 |   |—— repository
25 |   |—— mapper
26 |   └── dto
27 |
28 |—— project
29 |   |—— controller
30 |   |—— application
31 |   |—— domain
32 |   └── repository
```

```
33 |   ├── mapper
34 |   └── dto
35 |
36 | ├── template
37 |   ├── controller
38 |   ├── application
39 |   ├── domain
40 |   ├── repository
41 |   ├── mapper
42 |   └── parser
43 |
44 | ├── file
45 |   ├── controller
46 |   ├── application
47 |   ├── domain
48 |   ├── repository
49 |   ├── mapper
50 |   └── infra
51 |
52 | ├── orchestrator
53 |   ├── controller
54 |   ├── application
55 |   ├── domain
56 |   ├── worker
57 |   ├── repository
58 |   ├── mapper
59 |   └── event
60 |
61 | ├── agent
62 |   ├── application
63 |   ├── domain
64 |   ├── infra
65 |   └── dto
66 |
67 | ├── knowledge
68 |   ├── application
69 |   ├── domain
70 |   ├── infra
71 |   └── dto
72 |
73 | ├── render
74 |   ├── application
75 |   ├── domain
76 |   ├── infra
77 |   └── dto
78 |
79 | └── audit
```

```
80 |— controller
81 |— application
82 |— domain
83 |— repository
84 |— mapper
```

## 5.2 模块职责说明

| 模块           | 核心职责                        |
|--------------|-----------------------------|
| common       | 通用响应、异常处理、配置、工具类、枚举、安全基础能力  |
| auth         | 用户认证、角色权限、菜单权限、Token 校验     |
| report       | 报告记录、报告版本、预览、下载、分享、归档       |
| project      | 报告生成项目，管理模板、参考文档、Agent 参数   |
| template     | 模板上传、模板解析、模板变量、模板版本         |
| file         | 文件上传、下载、预签名 URL、MinIO 封装    |
| orchestrator | 报告生成任务编排、任务状态机、重试、日志        |
| agent        | 外部 Agent 调用适配、Agent 配置、调用日志 |
| knowledge    | 外部知识库检索适配、RAG 请求封装、检索日志     |
| render       | Word / PDF / Markdown 渲染导出  |
| audit        | 操作审计、Agent 调用审计、下载审计        |

## 5.3 模块调用规则

为避免模块化单体演变为混乱单体，需要遵守以下规则：

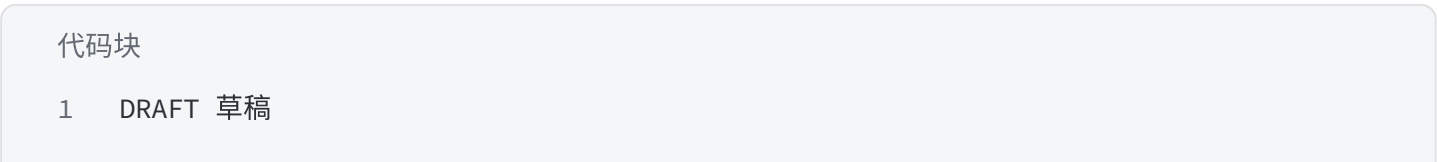
1. Controller 只能调用本模块 Application Service。
2. Application Service 可以调用其他模块暴露的 Facade 接口。
3. 不允许跨模块直接调用 Mapper。
4. 不允许跨模块直接操作对方数据库表。
5. Domain 层不依赖 Controller、Mapper、HTTP Client。
6. Infrastructure 层只能被 Application Service 或 Repository 调用。
7. common 模块只放通用能力，不放业务逻辑。

## 6. 核心业务流程设计

### 6.1 报告生成总体流程



### 6.2 报告生成状态机



```
2      ↓
3  SUBMITTED 已提交
4      ↓
5  GENERATING 生成中
6      ↓
7  KNOWLEDGE_RETRIEVING 知识库检索中
8      ↓
9  AGENT_PROCESSING Agent 处理中
10     ↓
11  RENDERING 渲染中
12     ↓
13  REVIEWING 待人工复核
14     ↓
15  COMPLETED 已完成
16
17  异常分支：
18  GENERATING / KNOWLEDGE_RETRIEVING / AGENT_PROCESSING / RENDERING
19     ↓
20  FAILED 生成失败
21     ↓
22  RETRYING 重试中
23     ↓
24  COMPLETED 或 FAILED
```

## 6.3 任务编排阶段

报告生成任务由 orchestrator 模块负责。

| 阶段      | 名称       | 说明                     |
|---------|----------|------------------------|
| Stage 1 | 初始化任务    | 加载报告项目、模板、用户参数         |
| Stage 2 | 文件准备     | 从 MinIO 获取模板和参考文档      |
| Stage 3 | 文档解析     | 提取参考文档文本、表格、图片等内容      |
| Stage 4 | 知识库检索    | 请求外部知识库获取标准条款、行业规范     |
| Stage 5 | Agent 生成 | 请求外部 Agent 生成章节内容和结论   |
| Stage 6 | 内容审查     | 请求外部 Agent 或规则服务进行合规校验 |
| Stage 7 | 文档渲染     | 根据模板渲染 Word / PDF      |
| Stage 8 | 结果存储     | 上传生成文件到 MinIO          |
| Stage 9 | 状态更新     | 更新报告版本状态和任务日志          |

## 7. 外部 Agent 集成设计

### 7.1 Agent 集成原则

系统不直接实现 Agent 能力，而是通过 agent 模块调用外部 Agent HTTP API。

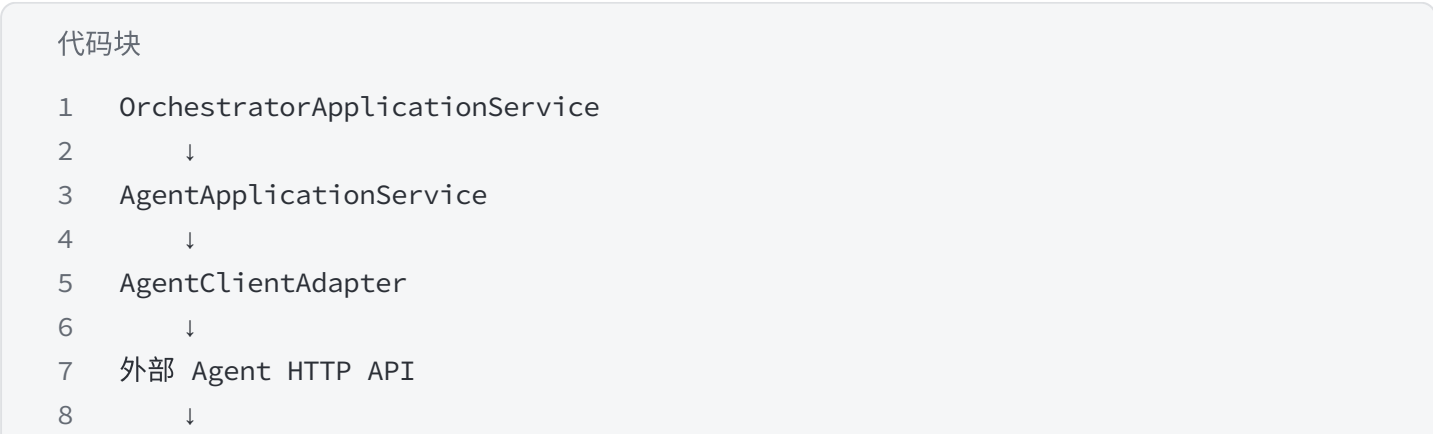
agent 模块需要统一处理：

- 1. Agent 地址配置。
- 2. 请求参数封装。
- 3. 认证信息注入。
- 4. 超时控制。
- 5. 重试控制。
- 6. 响应结果解析。
- 7. 调用日志记录。
- 8. 异常转换。

### 7.2 Agent 类型

| Agent 类型                | 说明               |
|-------------------------|------------------|
| CRYPTO_EVALUATION_AGENT | 密码安全应用测评报告 Agent |
| WATER_DOCUMENT_AGENT    | 水利文档生成 Agent     |
| GENERAL_REPORT_AGENT    | 通用报告 Agent       |
| COMPLIANCE_AUDIT_AGENT  | 合规审查 Agent       |

### 7.3 Agent 调用流程



```
9   返回结构化 JSON 结果
10  ↓
11  AgentResultParser
12  ↓
13  写入 agent_call_log
14  ↓
15  返回给 OrchestratorApplicationService
```

## 7.4 Agent 请求示例

代码块

```
1  {
2    "agentType": "CRYPTO_EVALUATION_AGENT",
3    "taskId": "T202606190001",
4    "input": {
5      "reportType": "密码安全应用测评报告",
6      "templateVariables": {},
7      "referenceDocuments": [],
8      "retrievedKnowledge": []
9    },
10   "outputFormat": "JSON"
11 }
```

## 7.5 Agent 返回示例

代码块

```
1  {
2    "success": true,
3    "sections": [
4      {
5        "sectionCode": "1.1",
6        "title": "测评对象概述",
7        "content": "....."
8      }
9    ],
10   "conclusion": "部分符合",
11   "risks": [],
12   "evidences": []
13 }
```

## 8. 外部知识库集成设计

### 8.1 知识库集成原则

系统不直接实现向量数据库和 RAG 检索能力，而是通过 knowledge 模块调用外部知识库 HTTP API。knowledge 模块负责统一封装知识库检索请求，并将检索结果注入 Agent 上下文。

### 8.2 知识库类型

| 知识库     | 说明                          |
|---------|-----------------------------|
| 密码标准知识库 | GM/T、等保 2.0、商用密码应用安全性评估相关标准 |
| 水利行业知识库 | SL / DL 系列水利行业规范            |
| 模板知识库   | 历史模板、报告结构、章节写作规范            |
| 术语知识库   | 专业术语、规范表达、禁用词               |

### 8.3 知识库调用流程

代码块

```
1  OrchestratorApplicationService
2      ↓
3  KnowledgeApplicationService
4      ↓
5  KnowledgeClientAdapter
6      ↓
7  外部知识库 HTTP API
8      ↓
9  返回标准条款 / 规范片段 / 参考内容
10     ↓
11  写入 knowledge_retrieval_log
12     ↓
13  注入 Agent 上下文
```

### 8.4 知识库请求示例

代码块

```
1  {
2      "domain": "CRYPTO",
3      "query": "身份鉴别 密码应用安全性评估 标准要求",
```

```
4     "topK": 5,  
5     "filters": {  
6         "standardCode": ["GM/T", "等保2.0"]  
7     }  
8 }
```

## 8.5 知识库返回示例

代码块

```
1  {  
2      "results": [  
3          {  
4              "title": "身份鉴别要求",  
5              "content": ".....",  
6              "source": "商用密码应用安全性评估标准",  
7              "score": 0.91  
8          }  
9      ]  
10 }
```

---

## 9. 数据架构设计

### 9.1 数据分类

| 数据类型       | 存储位置          | 说明                |
|------------|---------------|-------------------|
| 用户与权限数据    | MySQL         | 用户、角色、权限          |
| 报告元数据      | MySQL         | 报告、版本、状态、创建人      |
| 报告项目数据     | MySQL         | 项目配置、模板选择、参考文档关系  |
| 任务数据       | MySQL + Redis | 任务状态、阶段日志、重试次数    |
| 模板元数据      | MySQL         | 模板名称、类型、变量、版本     |
| 文件数据       | MinIO         | 模板文件、参考文档、生成报告    |
| Agent 调用日志 | MySQL         | 请求参数摘要、响应结果、耗时、状态 |
| 知识库检索日志    | MySQL         | 查询语句、召回内容、得分、耗时   |
| 审计日志       | MySQL         | 登录、上传、生成、下载、删除等操作 |

## 9.2 核心数据库表

### 9.2.1 report：报告表

| 字段                 | 类型       | 说明   |
|--------------------|----------|------|
| id                 | bigint   | 主键   |
| report_name        | varchar  | 报告名称 |
| report_type        | varchar  | 报告类型 |
| project_id         | bigint   | 所属项目 |
| creator_id         | bigint   | 创建人  |
| current_version_id | bigint   | 当前版本 |
| status             | varchar  | 报告状态 |
| created_at         | datetime | 创建时间 |
| updated_at         | datetime | 更新时间 |
| deleted            | tinyint  | 逻辑删除 |

### 9.2.2 report\_version：报告版本表

| 字段               | 类型       | 说明      |
|------------------|----------|---------|
| id               | bigint   | 主键      |
| report_id        | bigint   | 报告 ID   |
| version_no       | int      | 版本号     |
| file_id          | bigint   | 生成文件 ID |
| preview_file_id  | bigint   | 预览文件 ID |
| status           | varchar  | 版本状态    |
| generate_task_id | bigint   | 生成任务 ID |
| created_at       | datetime | 创建时间    |

9.2.3 report\_project：报告生成项目表

| 字段           | 类型       | 说明       |
|--------------|----------|----------|
| id           | bigint   | 主键       |
| project_name | varchar  | 项目名称     |
| report_type  | varchar  | 报告类型     |
| template_id  | bigint   | 模板 ID    |
| agent_type   | varchar  | Agent 类型 |
| creator_id   | bigint   | 创建人      |
| config_json  | json     | 项目配置     |
| status       | varchar  | 项目状态     |
| created_at   | datetime | 创建时间     |
| updated_at   | datetime | 更新时间     |

9.2.4 template：模板表

| 字段            | 类型       | 说明      |
|---------------|----------|---------|
| id            | bigint   | 主键      |
| template_name | varchar  | 模板名称    |
| report_type   | varchar  | 适用报告类型  |
| file_id       | bigint   | 模板文件 ID |
| version_no    | int      | 模板版本    |
| status        | varchar  | 启用 / 停用 |
| description   | text     | 描述      |
| created_by    | bigint   | 创建人     |
| created_at    | datetime | 创建时间    |

9.2.5 template\_variable：模板变量表

| 字段            | 类型      | 说明    |
|---------------|---------|-------|
| id            | bigint  | 主键    |
| template_id   | bigint  | 模板 ID |
| variable_key  | varchar | 变量名   |
| variable_name | varchar | 中文名称  |
| default_value | varchar | 默认值   |
| required      | tinyint | 是否必填  |
| data_type     | varchar | 数据类型  |

9.2.6 file\_object：文件对象表

| 字段            | 类型       | 说明           |
|---------------|----------|--------------|
| id            | bigint   | 主键           |
| bucket_name   | varchar  | MinIO bucket |
| object_key    | varchar  | 对象路径         |
| original_name | varchar  | 原始文件名        |
| file_type     | varchar  | 文件类型         |
| file_size     | bigint   | 文件大小         |
| content_type  | varchar  | MIME 类型      |
| checksum      | varchar  | 文件校验值        |
| created_by    | bigint   | 上传人          |
| created_at    | datetime | 上传时间         |

9.2.7 generate\_task：生成任务表

| 字段                | 类型       | 说明      |
|-------------------|----------|---------|
| id                | bigint   | 主键      |
| report_id         | bigint   | 报告 ID   |
| report_version_id | bigint   | 报告版本 ID |
| task_status       | varchar  | 任务状态    |
| current_stage     | varchar  | 当前阶段    |
| retry_count       | int      | 重试次数    |
| error_message     | text     | 错误信息    |
| started_at        | datetime | 开始时间    |
| finished_at       | datetime | 结束时间    |

9.2.8 task\_stage\_log：任务阶段日志表

| 字段             | 类型       | 说明    |
|----------------|----------|-------|
| id             | bigint   | 主键    |
| task_id        | bigint   | 任务 ID |
| stage_name     | varchar  | 阶段名称  |
| status         | varchar  | 状态    |
| input_summary  | text     | 输入摘要  |
| output_summary | text     | 输出摘要  |
| error_message  | text     | 错误信息  |
| cost_ms        | bigint   | 耗时    |
| created_at     | datetime | 创建时间  |

9.2.9 agent\_call\_log：Agent 调用日志表

| 字段            | 类型       | 说明       |
|---------------|----------|----------|
| id            | bigint   | 主键       |
| task_id       | bigint   | 任务 ID    |
| agent_type    | varchar  | Agent 类型 |
| request_id    | varchar  | 外部请求 ID  |
| request_body  | text     | 请求参数     |
| response_body | text     | 返回结果     |
| success       | tinyint  | 是否成功     |
| cost_ms       | bigint   | 调用耗时     |
| created_at    | datetime | 创建时间     |

9.2.10 knowledge\_retrieval\_log：知识库检索日志表

| 字段            | 类型       | 说明    |
|---------------|----------|-------|
| id            | bigint   | 主键    |
| task_id       | bigint   | 任务 ID |
| domain        | varchar  | 领域    |
| query_text    | text     | 查询内容  |
| top_k         | int      | 返回数量  |
| response_body | text     | 检索结果  |
| cost_ms       | bigint   | 耗时    |
| created_at    | datetime | 创建时间  |

## 10. 文件存储架构设计

### 10.1 MinIO Bucket 规划

| Bucket        | 存储内容                          |
|---------------|-------------------------------|
| dgp-template  | Word 模板文件                     |
| dgp-reference | 用户上传的参考文档                     |
| dgp-report    | 生成后的 Word / PDF / Markdown 报告 |
| dgp-preview   | 在线预览转换后的文件                    |
| dgp-temp      | 临时中间文件                        |

### 10.2 文件路径规则

代码块

```
1  /templates/{reportType}/{templateId}/{versionNo}/{filename}
2
3  /references/{userId}/{projectId}/{fileId}/{filename}
4
5  /reports/{reportType}/{reportId}/{versionNo}/{filename}
6
```

### 10.3 文件访问控制

- 1. 文件不直接暴露真实 MinIO 地址。
- 2. 后端根据权限生成临时预签名 URL。
- 3. 预签名 URL 设置有效期，例如 10 分钟。
- 4. 下载行为写入审计日志。
- 5. 敏感报告可禁止分享或限制下载。
- 6. 删除文件时优先逻辑删除元数据，物理文件可由定时清理任务处理。

## 11. 接口架构设计

### 11.1 统一响应结构

代码块

```
1  {
2    "code": 0,
3    "message": "success",
4    "data": {},
5    "requestId": "REQ202606190001"
6  }
```

### 11.2 报告项目接口

| 方法     | 路径                 | 说明       |
|--------|--------------------|----------|
| POST   | /api/projects      | 创建报告生成项目 |
| GET    | /api/projects/{id} | 查询项目详情   |
| PUT    | /api/projects/{id} | 修改项目配置   |
| DELETE | /api/projects/{id} | 删除项目     |

### 11.3 报告生成接口

| 方法     | 路径                           | 说明       |
|--------|------------------------------|----------|
| POST   | /api/reports/generate        | 提交报告生成任务 |
| GET    | /api/reports/{id}            | 查询报告详情   |
| GET    | /api/reports/{id}/versions   | 查询报告版本   |
| GET    | /api/reports/{id}/preview    | 在线预览报告   |
| GET    | /api/reports/{id}/download   | 下载报告     |
| POST   | /api/reports/{id}/regenerate | 重新生成报告   |
| DELETE | /api/reports/{id}            | 删除报告     |

## 11.4 模板管理接口

| 方法   | 路径                            | 说明      |
|------|-------------------------------|---------|
| POST | /api/templates/upload         | 上传模板    |
| GET  | /api/templates                | 查询模板列表  |
| GET  | /api/templates/{id}           | 查询模板详情  |
| GET  | /api/templates/{id}/variables | 查询模板变量  |
| GET  | /api/templates/{id}/download  | 下载模板    |
| PUT  | /api/templates/{id}/status    | 启用或停用模板 |

## 11.5 文件接口

| 方法   | 路径                           | 说明     |
|------|------------------------------|--------|
| POST | /api/files/upload            | 上传文件   |
| GET  | /api/files/{id}/download-url | 获取下载地址 |
| GET  | /api/files/{id}/preview-url  | 获取预览地址 |

## 11.6 任务接口

| 方法   | 路径                     | 说明     |
|------|------------------------|--------|
| GET  | /api/tasks/{id}        | 查询任务详情 |
| GET  | /api/tasks/{id}/logs   | 查询任务日志 |
| POST | /api/tasks/{id}/retry  | 重试任务   |
| POST | /api/tasks/{id}/cancel | 取消任务   |

## 12. 异步任务与并发设计

### 12.1 异步任务方案

一期可以使用 Redis Stream 或 Redis List 实现轻量任务队列。

代码块

```
1  用户提交生成任务
2      ↓
3  写入 MySQL generate_task
4      ↓
5  写入 Redis 任务队列
6      ↓
7  后台 Worker 消费任务
8      ↓
9  执行任务阶段
10     ↓
11 更新 MySQL 任务状态
```

### 12.2 Worker 设计

系统内部启动后台任务消费者，负责扫描和执行生成任务。

代码块

```
1  @Component
2  public class ReportGenerateWorker {
3
4      @Scheduled(fixedDelay = 1000)
5      public void pollTask() {
6          // 1. 从 Redis 队列拉取任务
7          // 2. 获取 Redis 分布式锁
```

```
8      // 3. 校验任务状态
9      // 4. 执行 Orchestrator
10     // 5. 更新任务状态
11     // 6. 记录阶段日志
12 }
13 }
```

## 12.3 并发控制

| 场景            | 控制方式             |
|---------------|------------------|
| 同时生成报告数量      | 线程池限制最大并发        |
| 同一报告重复生成      | Redis 分布式锁       |
| 外部 Agent 调用过载 | 限流 + 超时控制        |
| 大文件上传         | 文件大小限制 + 分片上传    |
| 任务重复消费        | 任务状态幂等判断         |
| 多应用副本执行任务     | Redis 分布式锁防止重复执行 |

# 13. 缓存设计

## 13.1 Redis 缓存内容

| Key                       | 说明       | TTL   |
|---------------------------|----------|-------|
| template:{id}             | 模板元数据    | 30 分钟 |
| template:variables:{id}   | 模板变量     | 30 分钟 |
| user:permission:{userId}  | 用户权限     | 10 分钟 |
| agent:profile:{agentType} | Agent 配置 | 10 分钟 |
| knowledge:config:{domain} | 知识库配置    | 10 分钟 |
| task:lock:{taskId}        | 任务执行锁    | 5 分钟  |

## 13.2 缓存一致性

1. 模板更新后删除模板缓存。
2. 用户权限变更后删除权限缓存。
3. Agent 配置变更后删除 Agent 缓存。
4. 知识库配置变更后删除知识库配置缓存。
5. 缓存只作为加速，不作为最终数据源。
6. 数据最终以 MySQL 为准。

## 14. 安全架构设计

### 14.1 认证与授权

系统支持 SSO 接入，后端使用 JWT 标识用户身份。

权限模型采用 RBAC。

| 角色    | 权限                  |
|-------|---------------------|
| 平台管理员 | 用户管理、模板管理、全部报告、系统配置 |
| 模板管理员 | 模板上传、下载、启停、版本管理     |
| 报告创建者 | 创建报告、生成报告、查看自己的报告   |
| 报告查看者 | 查看和下载授权报告           |

### 14.2 数据安全

1. 全站 HTTPS。
2. 敏感字段脱敏展示。
3. 文件下载使用预签名 URL。
4. 不同部门数据逻辑隔离。
5. 重要操作写入审计日志。
6. Agent 请求中避免传递无关敏感信息。
7. 对外部 Agent 和知识库调用设置访问白名单。
8. 外部接口调用密钥统一放在配置中心或环境变量中，不写入代码。

### 14.3 审计范围

需要审计的操作包括：

- 1. 用户登录。
- 2. 上传模板。
- 3. 上传参考文档。
- 4. 创建报告。
- 5. 生成报告。
- 6. 下载报告。
- 7. 分享报告。
- 8. 删除报告。
- 9. 调用外部 Agent。
- 10. 调用外部知识库。

## 15. 可观测性设计

### 15.1 日志设计

系统日志分为：

| 日志类型          | 说明          |
|---------------|-------------|
| access log    | HTTP 请求访问日志 |
| biz log       | 业务操作日志      |
| task log      | 报告生成任务日志    |
| agent log     | Agent 调用日志  |
| knowledge log | 知识库检索日志     |
| error log     | 异常日志        |
| audit log     | 审计日志        |

### 15.2 Traceld 设计

每次请求生成 requestId。

每个报告生成任务生成 taskId。

Agent 调用和知识库调用都需要携带 requestId 和 taskId，方便全链路追踪。

代码块

1    requestId → reportId → taskId → agentCallId → fileId

### 15.3 监控指标

| 指标                            | 说明            |
|-------------------------------|---------------|
| report_generate_total         | 报告生成总数        |
| report_generate_success_total | 成功生成数量        |
| report_generate_tailed_total  | 失败生成数量        |
| report_generate_p95_cost      | 报告生成 P95 耗时   |
| agent_call_cost               | Agent 调用耗时    |
| knowledge_search_cost         | 知识库检索耗时       |
| file_upload_cost              | 文件上传耗时        |
| task_queue_size               | 任务队列积压数量      |
| redis_lock_failed_total       | Redis 锁获取失败次数 |
| minio_upload_failed_total     | MinIO 上传失败次数  |

## 16. 异常与容错设计

### 16.1 异常分类

| 异常类型          | 处理方式        |
|---------------|-------------|
| 参数错误          | 返回 40001    |
| 未登录或 Token 失效 | 返回 40101    |
| 权限不足          | 返回 40301    |
| 模板不存在         | 返回 40401    |
| 文件不存在         | 返回 40402    |
| Agent 调用失败    | 记录日志，进入重试   |
| 知识库调用失败       | 记录日志，允许重试   |
| 渲染失败          | 标记任务失败，可重试  |
| MinIO 上传失败    | 重试后失败       |
| MySQL 异常      | 回滚事务，返回系统错误 |
| Redis 异常      | 降级或阻断任务执行   |

## 16.2 重试策略

| 操作         | 重试次数 |
|------------|------|
| Agent 调用   | 3 次  |
| 知识库检索      | 2 次  |
| MinIO 上传   | 3 次  |
| 文档渲染       | 1 次  |
| Redis 任务消费 | 幂等重试 |

## 16.3 幂等设计

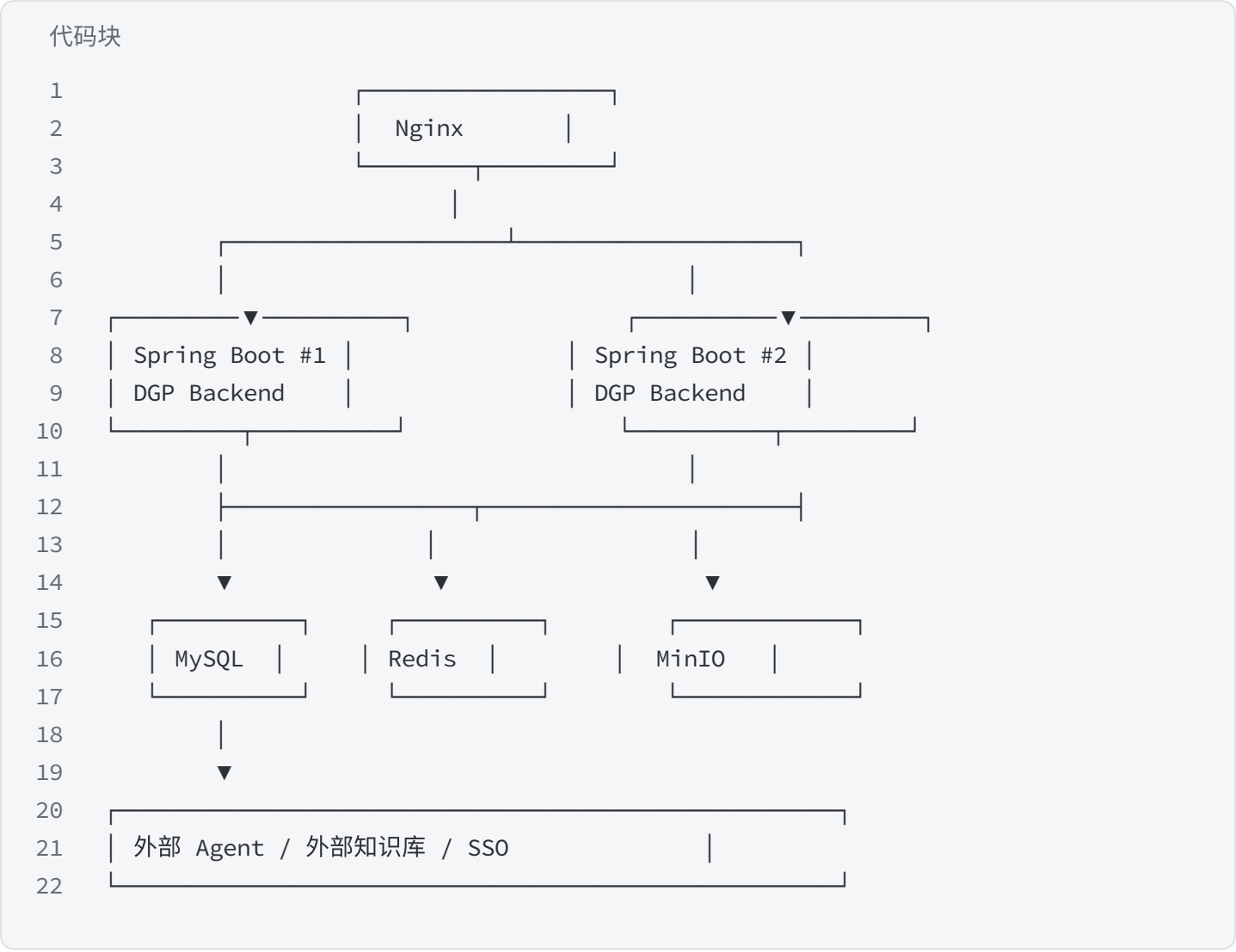
以下接口需要支持幂等：

- 提交报告生成任务。
- 重试报告生成任务。
- 文件上传完成回调。
- Agent 调用结果写入。

- 5. 知识库检索结果写入。
- 6. 报告版本状态更新。

## 17. 部署架构设计

### 17.1 部署拓扑



### 17.2 部署说明

| 组件             | 部署方式              |
|----------------|-------------------|
| Spring Boot 后端 | Docker 容器，可部署多个副本 |
| MySQL          | 主从或云数据库           |
| Redis          | 主从 / 哨兵 / 云 Redis |
| MinIO          | 单节点或分布式部署         |
| Nginx          | 统一入口、HTTPS 终止     |
| 外部 Agent       | 独立提供 HTTP API     |
| 外部知识库          | 独立提供 HTTP API     |

### 17.3 多副本注意事项

由于 Spring Boot 后端可以部署多个副本，所以需要注意：

- 1. 文件不能存在本地磁盘，必须进入 MinIO。
- 2. 任务锁不能使用本地锁，需要使用 Redis 分布式锁。
- 3. 定时任务需要防止多实例重复执行。
- 4. 用户 Session 不放本地内存，使用 JWT 或 Redis。
- 5. 报告生成任务状态以 MySQL 为准。
- 6. Worker 消费任务时必须做任务状态校验，防止重复执行。

## 18. 模块治理规范

为了保证模块化单体长期可维护，需要制定模块治理规范。

### 18.1 包依赖规范

- 1. common 可以被所有模块依赖。
- 2. 各业务模块之间不得直接访问对方 mapper。
- 3. 各业务模块之间通过 application 层接口或 facade 接口交互。
- 4. domain 层不得依赖 controller、mapper、infra。
- 5. infra 层负责封装技术细节，业务逻辑不得下沉到 infra 层。

### 18.2 代码规范

- 1. Controller 只做参数接收和响应封装。
- 2. Application Service 负责编排业务流程。
- 3. Domain 负责表达业务规则。
- 4. Mapper 只负责数据访问。
- 5. DTO、DO、Domain Object 不混用。
- 6. 外部接口请求和响应对象单独定义。
- 7. 所有关键操作必须记录操作日志。

### 18.3 配置规范

- 1. Agent 地址、密钥、超时时间通过配置管理。
- 2. 知识库地址、密钥、topK 默认值通过配置管理。
- 3. MinIO bucket、访问密钥通过配置管理。
- 4. Redis key 前缀统一管理。
- 5. 不同环境使用不同 Spring Profile，例如 dev、test、prod。

## 19. 架构风险与应对

| 风险           | 说明           | 应对                    |
|--------------|--------------|-----------------------|
| 外部 Agent 不稳定 | Agent 响应慢或失败 | 超时、重试、失败记录            |
| 知识库召回不准确     | 影响报告专业性      | 保存检索日志，支持人工复核         |
| 报告生成耗时过长     | 超过 30 秒目标    | 异步任务、阶段并行、缓存          |
| 模板格式复杂       | Word 模板解析失败  | 模板上传时预校验              |
| 文件过大         | 上传和预览慢       | 限制大小、压缩、分片            |
| 单体代码膨胀       | 后期维护困难       | 严格模块边界，禁止跨模块调用 Mapper |
| 多副本重复消费任务    | 生成重复报告       | Redis 分布式锁 + 任务幂等     |
| 外部接口返回结构变化   | 导致解析失败       | Adapter 层统一适配，返回结构版本化 |

## 20. 技术选型总结

| 类型       | 技术                                           |
|----------|----------------------------------------------|
| 后端语言     | Java                                         |
| Web 框架   | Spring Boot                                  |
| ORM      | MyBatis / MyBatis-Plus                       |
| 数据库      | MySQL                                        |
| 缓存       | Redis                                        |
| 异步队列     | Redis Stream / Redis List                    |
| 文件存储     | MinIO                                        |
| 文档处理     | Apache POI / docx4j                          |
| PDF 转换   | LibreOffice Headless / OnlyOffice / PDF 渲染服务 |
| HTTP 客户端 | OpenFeign / WebClient / OkHttp               |
| 安全       | Spring Security / JWT / SSO                  |
| 部署       | Docker / Kubernetes                          |
| 监控       | Prometheus / Grafana                         |
| 日志       | Logback / ELK                                |

## 21. 总结

本系统采用模块化单体架构，在单个 Spring Boot 应用内按照领域模块拆分，实现报告生成平台的核心能力。

该架构具备以下特点：

1. 开发和部署成本较低，适合项目一期快速落地。
2. 模块边界清晰，避免形成混乱单体。
3. Agent 能力和知识库能力均通过外部 HTTP 接口接入。
4. 平台本身专注于报告项目管理、任务编排、模板管理、文件管理、报告生命周期管理和审计追踪。
5. 通过 MySQL、Redis、MinIO 形成稳定的数据、缓存、队列和文件支撑。
6. 通过任务状态机、阶段日志、失败重试和幂等控制保障报告生成过程可控。
7. 通过 RBAC、审计日志、预签名 URL 和敏感字段保护保障系统安全。

因此，本架构能够满足报告生成平台一期建设需求，并为后续功能扩展、多行业接入和复杂报告生成场景提供稳定基础。